

Introduction to Machine Learning (25737-2)

Problem Set 5

Spring Semester 1403-04

Department of Electrical Engineering

Sharif University of Technology

Instructor: Dr. S. Amini

Due on 16 Khordad, 1404 at 23:59



1 Kernel Ridge Regression

In contrast to ordinary least squares which has a cost function

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2,$$

we can also add a term that penalizes large weights in θ . In ridge regression, our least squares cost is regularized by adding a term $\frac{\lambda}{2} \|\theta\|^2$, where $\lambda > 0$ is a fixed (known) constant (regularization will be discussed at greater length in an upcoming course lecture). The ridge regression cost function is then

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2 + \frac{\lambda}{2} \|\theta\|^2.$$

- (a) Use the vector notation described in class to find a closed-form expression for the value of θ which minimizes the ridge regression cost function.
- (b) Suppose that we want to use kernels to implicitly represent our feature vectors in a high-dimensional (possibly infinite-dimensional) space. Using a feature mapping ϕ , the ridge regression cost function becomes

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (\theta^T \phi(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2} \|\theta\|^2.$$

Making a prediction on a new input x_{new} would now be done by computing $\theta^T \phi(x_{\text{new}})$. Show how we can use the “kernel trick” to obtain a closed form for the prediction on the new input without ever explicitly computing $\phi(x_{\text{new}})$. You may assume that the parameter vector θ can be expressed as $\theta = \sum_{i=1}^m \alpha_i \phi(x^{(i)})$. [Hint: You may find the following identity useful: $(\lambda I + BA)^{-1}B = B(\lambda I + AB)^{-1}$.]

Answer

- (a) Using the design matrix notation, we can rewrite $J(\theta)$ as

$$J(\theta) = \frac{1}{2}(X\theta - \mathbf{y})^T(X\theta - \mathbf{y}) + \frac{\lambda}{2}\theta^T\theta.$$

Then the gradient is

$$\nabla_{\theta}J(\theta) = X^T X\theta - X^T \mathbf{y} + \lambda\theta.$$

Setting the gradient to 0 gives us

$$0 = X^T X\theta - X^T \mathbf{y} + \lambda\theta \implies \theta = (X^T X + \lambda I)^{-1} X^T \mathbf{y}.$$

(b) Let Φ be the design matrix associated with the feature vectors $\phi(x^{(i)})$. Then

$$\theta = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T \mathbf{y} = \Phi^T (\Phi \Phi^T + \lambda I)^{-1} \mathbf{y} = \Phi^T (K + \lambda I)^{-1} \mathbf{y},$$

where K is the kernel matrix for the training set ($K_{ij} = \phi(x^{(i)})^T \phi(x^{(j)})$). To predict a new value y_{new} , we compute

$$\hat{y}_{\text{new}} = \theta^T \phi(x_{\text{new}}) = \mathbf{y}^T (K + \lambda I)^{-1} \Phi \phi(x_{\text{new}}) = \sum_{i=1}^m \alpha_i K(x^{(i)}, x_{\text{new}}),$$

where $\alpha = (K + \lambda I)^{-1} \mathbf{y}$. All these terms can be efficiently computed using the kernel function.

2 Fisher kernel

Consider a Gaussian distribution with mean vector μ and known covariance matrix Σ . The *Fisher kernel* between two data points x and x' is defined as

$$k(x, x') = U(x)^{\top} \mathcal{I}^{-1} U(x'),$$

where $U(x) = \nabla_{\mu} \log p(x | \mu)$ is the gradient of the log-likelihood with respect to the mean parameter μ , and $\mathcal{I}$ is the Fisher information matrix.

Derive the Fisher kernel for this Gaussian distribution. First, compute the Fisher score:

$$U(x) = \nabla_{\mu} \log \mathcal{N}(x | \mu, \Sigma)$$

Next, compute the Fisher information matrix:

$$\mathcal{I} = \mathbb{E}_x [U(x)U(x)^{\top}]$$

Substitute these into the Fisher kernel expression and show that it simplifies to:

$$k(x, x') = (x - \mu)^{\top} \Sigma^{-1} (x' - \mu),$$

which is the squared Mahalanobis inner product between x and x' .

Answer

In order to evaluate the Fisher kernel for a Gaussian with fixed covariance S , we treat only the mean vector μ as the parameter. The Fisher score is the gradient of the log-likelihood with respect to μ :

$$g(\mu, \mathbf{x}) = \nabla_{\mu} \ln \left[(2\pi)^{-\frac{d}{2}} |S|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^{\top} S^{-1}(\mathbf{x} - \mu)\right) \right] = S^{-1}(\mathbf{x} - \mu).$$

The Fisher information matrix is the expected outer product of the score:

$$F = \mathbb{E}_{\mathbf{x}}[g(\mu, \mathbf{x}) g(\mu, \mathbf{x})^{\top}] = S^{-1} \mathbb{E}_{\mathbf{x}}[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^{\top}] S^{-1}.$$

Since for a Gaussian $\mathbb{E}_{\mathbf{x}}[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^{\top}] = S$, it follows that

$$F = S^{-1}.$$

Finally, the Fisher kernel itself is

$$k(\mathbf{x}, \mathbf{x}') = g(\mu, \mathbf{x})^{\top} F^{-1} g(\mu, \mathbf{x}') = (\mathbf{x} - \mu)^{\top} S^{-1} (\mathbf{x}' - \mu),$$

which is exactly the squared Mahalanobis distance between $\mathbf{x}$ and $\mathbf{x}'$.

3 ℓ_2 -Norm Soft-Margin SVM

In class, we saw that if our data is not linearly separable, then we need to modify our support vector machine algorithm by introducing an error margin that must be minimized. Specifically, the formulation we have looked at is known as the ℓ_2 soft-margin SVM. This new algorithm is given by the following optimization problem (notice that the slack penalties are now squared):

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + \frac{C}{2} \sum_{i=1}^m \xi_i^2 \quad \text{s.t.} \quad y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad i = 1, \dots, m.$$

- (a) Notice that we have dropped the $\xi_i \geq 0$ constraint in the ℓ_2 problem. Show that these non-negativity constraints can be removed without changing the optimal value.
- (b) What is the Lagrangian of the ℓ_2 soft-margin SVM optimization problem?
- (c) Minimize the Lagrangian with respect to w , b , and ξ by taking the following gradients: $\nabla_w L$, $\partial L / \partial b$, and $\nabla_{\xi} L$, and then setting them equal to zero. Here $\xi = [\xi_1, \dots, \xi_m]^T$.
- (d) What is the dual of the ℓ_2 soft-margin SVM optimization problem?

Answer

- (a) Consider a potential solution with some $\xi_i < 0$. Then the constraint $y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i$ would still be satisfied by setting $\xi_i = 0$, which also decreases the objective, so any optimal solution has $\xi_i \geq 0$.
- (b)

$$L(w, b, \xi, \alpha) = \frac{1}{2} w^T w + \frac{C}{2} \sum_{i=1}^m \xi_i^2 - \sum_{i=1}^m \alpha_i [y^{(i)}(w^T x^{(i)} + b) - 1 + \xi_i],$$

where $\alpha_i \geq 0$.

(c)

$$\begin{aligned}\nabla_w L = w - \sum_{i=1}^m \alpha_i y^{(i)} x^{(i)} = 0 &\implies w = \sum_{i=1}^m \alpha_i y^{(i)} x^{(i)}, \\ \frac{\partial L}{\partial b} = - \sum_{i=1}^m \alpha_i y^{(i)} = 0, \\ \nabla_\xi L = C\xi - \alpha = 0 &\implies C\xi_i = \alpha_i, \quad i = 1, \dots, m.\end{aligned}$$

(d) The objective function for the dual is

$$\begin{aligned}W(\alpha) &= \min_{w, b, \xi} \mathcal{L}(w, b, \xi, \alpha) \\ &= \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m (\alpha_i y^{(i)} x^{(i)})^T (\alpha_j y^{(j)} x^{(j)}) + \frac{1}{2} \sum_{i=1}^m \frac{\alpha_i}{\xi_i} \xi_i^2 \\ &\quad - \sum_{i=1}^m \alpha_i \left[y^{(i)} \left(\left(\sum_{j=1}^m \alpha_j y^{(j)} x^{(j)} \right)^T x^{(i)} + b \right) - 1 + \xi_i \right] \\ &= -\frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} (x^{(i)})^T x^{(j)} + \frac{1}{2} \sum_{i=1}^m \alpha_i \xi_i \\ &\quad - \left(\sum_{i=1}^m \alpha_i y^{(i)} \right) b + \sum_{i=1}^m \alpha_i - \sum_{i=1}^m \alpha_i \xi_i \\ &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} (x^{(i)})^T x^{(j)} - \frac{1}{2} \sum_{i=1}^m \alpha_i \xi_i \\ &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} (x^{(i)})^T x^{(j)} - \frac{1}{2} \sum_{i=1}^m \frac{\alpha_i^2}{C}\end{aligned}$$

Then the dual formulation of our problem is

$$\begin{aligned}\max_{\alpha} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} (x^{(i)})^T x^{(j)} - \frac{1}{2} \sum_{i=1}^m \frac{\alpha_i^2}{C} \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, \dots, m \\ & \sum_{i=1}^m \alpha_i y^{(i)} = 0\end{aligned}$$

4 SVM with Gaussian Kernel

Consider the task of training a support vector machine using the Gaussian kernel

$$K(x, z) = \exp(-\|x - z\|^2 / \tau^2).$$

We will show that as long as there are no two identical points in the training set, we can always find a value for the bandwidth parameter τ such that the SVM achieves zero training error.

- (a) Recall from class that the decision function learned by the support vector machine can be written

$$f(x) = \sum_{i=1}^m \alpha_i y^{(i)} K(x^{(i)}, x) + b.$$

Assume that the training data $\{(x^{(i)}, y^{(i)})\}_{i=1}^m$ consists of points separated by at least a distance ε , i.e. $\|x^{(i)} - x^{(j)}\| \geq \varepsilon$ for $i \neq j$. Find values for the parameters $\{\alpha_i\}$ and b , and a choice of τ , such that each $x^{(i)}$ is correctly classified.

Hint: Let $\alpha_i = 1$ for all i and $b = 0$. Then note that for $y \in \{\pm 1\}$ the prediction on $x^{(i)}$ is correct if $|f(x^{(i)}) - y^{(i)}| < 1$, so choose τ to satisfy that inequality for all i .

- (b) Suppose we run a SVM with slack variables using the parameter τ you found in part (a). Will the resulting classifier necessarily obtain zero training error? Why or why not? A short explanation (without proof) will suffice.

Answer

- (a) First we set $\alpha_i = 1$ for all $i = 1, \dots, m$ and $b = 0$. Then, for a training example $\{x^{(i)}, y^{(i)}\}$, we get

$$\begin{aligned} |f(x^{(i)}) - y^{(i)}| &= \left| \sum_{j=1}^m y^{(j)} K(x^{(j)}, x^{(i)}) - y^{(i)} \right| \\ &= \left| \sum_{j=1}^m y^{(j)} \exp(-\|x^{(j)} - x^{(i)}\|^2 / \tau^2) - y^{(i)} \right| \\ &= \left| y^{(i)} + \sum_{j \neq i} y^{(j)} \exp(-\|x^{(j)} - x^{(i)}\|^2 / \tau^2) - y^{(i)} \right| \\ &= \left| \sum_{j \neq i} y^{(j)} \exp(-\|x^{(j)} - x^{(i)}\|^2 / \tau^2) \right| \\ &\leq \sum_{j \neq i} |y^{(j)} \exp(-\|x^{(j)} - x^{(i)}\|^2 / \tau^2)| \\ &= \sum_{j \neq i} |y^{(j)}| \cdot \exp(-\|x^{(j)} - x^{(i)}\|^2 / \tau^2) \\ &= \sum_{j \neq i} \exp(-\|x^{(j)} - x^{(i)}\|^2 / \tau^2) \\ &\leq \sum_{j \neq i} \exp(-\varepsilon^2 / \tau^2) \\ &= (m - 1) \exp(-\varepsilon^2 / \tau^2) \end{aligned}$$

The first inequality comes from repeated application of the triangle inequality $|a + b| \leq |a| + |b|$, and the second inequality (1) from the assumption that $\|x^{(j)} - x^{(i)}\| \geq \varepsilon$ for all $i \neq j$. Thus we need to choose a τ such that

$$(m - 1) \exp(-\varepsilon^2 / \tau^2) < 1,$$

or

$$\tau < \frac{\epsilon}{\log(m-1)}.$$

By choosing, for example, $\tau = \epsilon / \log m$, we are done.

- (b) The classifier will obtain zero training error. The SVM without slack variables will always return zero training error if it is able to find a solution, so all that remains to be shown is that there exists at least one feasible point.

Consider the constraint $y^{(i)}(w^T x^{(i)} + b)$ for some i , and let $b = 0$. Then

$$y^{(i)}(w^T x^{(i)} + b) = y^{(i)} \cdot f(x^{(i)}) > 0$$

since $f(x^{(i)})$ and $y^{(i)}$ have the same sign, and shown above. Therefore, as we choose all the α_i 's large enough, $y^{(i)}(w^T x^{(i)} + b) > 1$, so the optimization problem is feasible.

5 Decision Trees

Master Yoda is concerned about the number of Jedi apprentices that have turned to the Dark Side, so he's decided to train a decision tree on some historical data to help identify problem cases in the future. The following table summarizes whether or not each of 12 initiates turned to the Dark Side based on their age when their Jedi training began, whether or not they completed their training, their general disposition, and their species.

Dark Side	Age Started Training	Completed Training	Disposition	Species
0	5	1	Happy	Human
0	9	1	Happy	Gungan
0	6	0	Happy	Wookiee
0	6	1	Sad	Mon Calamari
0	7	0	Sad	Human
0	8	1	Angry	Human
0	5	1	Angry	Ewok
1	9	0	Happy	Ewok
1	8	0	Sad	Human
1	8	0	Sad	Human
1	6	0	Angry	Wookiee
1	7	0	Angry	Mon Calamari

- (a) What is the initial entropy of Dark Side?

Answer:

$$H = -\frac{5}{12} \log_2 \frac{5}{12} - \frac{7}{12} \log_2 \frac{7}{12} = \boxed{0.9799}$$

- (b) Which attribute would the decision-tree building algorithm choose to use for the root of the tree?

Answer: Completed Training

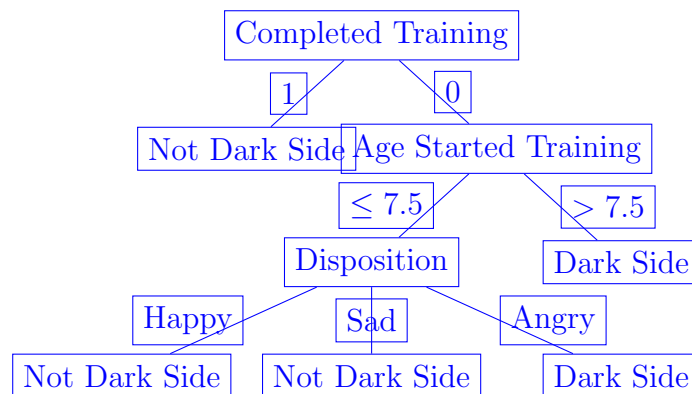
- (c) What is the information gain of the attribute you chose to split on in the previous question?

Answer:

$$\text{Gain} = 0.9799 - \left(\frac{5}{12}(0) + \frac{7}{12} \left(-\frac{5}{7} \log_2 \frac{5}{7} - \frac{2}{7} \log_2 \frac{2}{7} \right) \right) = \boxed{0.4764}$$

- (d) Draw the full decision tree that would be learned for this data (with no pruning).

Answer:



- (e) Consider noisy data: which test case would you be most confident about? Why?

Name	Age Started Training	Completed Training	Disposition	Species
Ardath	5	0	Angry	Human
Barbar	8	0	Angry	Gungan
Caldar	8	0	Happy	Mon Calamari

Answer: *Barbar*. He did not complete training, is old, and is angry. Even if one rule is incorrect, the other suffices for predicting Dark Side. Most confident in Barbar.

- (f) What would be the training set error for this dataset? (With no pruning)

Z	A	B	C
0	0	0	0
0	0	0	1
0	0	0	1
0	0	1	0
0	0	1	1
1	0	1	1
0	1	0	0
1	1	0	1
1	1	1	0
1	1	1	0
0	1	1	1
1	1	1	1

Answer: There are 4 duplicate input rows; 2 have contradictory outputs. 2 records are always misclassified.

- (g) What is the maximum training set error (as a fraction) a decision tree could have if the output can take k values?

Answer:

$$\frac{k-1}{k}$$

6 More on Decision Trees

The following questions refer to the deeper structure and learning dynamics of decision trees and ensemble methods.

- (a) Describe at least two regularization techniques used to avoid overfitting in decision trees. How does pruning differ from early stopping during tree construction?

Answer: Common techniques:

- Limiting maximum depth or number of leaves.
- Minimum samples required to split or be in a leaf.

Early stopping prevents further splits during training, while pruning removes unnecessary branches after a full tree is grown, based on performance criteria.

- (b) A common strategy for constructing decision trees is to begin with a single root node labeled by the majority class, and then iteratively split nodes. At each step, the algorithm evaluates the effect of splitting a leaf node using a "gain" function, which measures the improvement resulting from the split. The algorithm selects the feature that maximizes this gain or chooses not to split if no improvement is substantial.

Name and describe two commonly used gain functions in decision tree algorithms. How does each function evaluate the quality of a split?

Answer: Two commonly used gain functions are:

- **Information Gain:** Measures the reduction in entropy after a split. Entropy quantifies uncertainty in class labels. The higher the reduction in entropy, the better the split. Defined as:

$$C(a) = -a \log a - (1 - a) \log(1 - a)$$

- **Gini Index:** Measures impurity in the dataset. A lower Gini index after a split indicates a purer separation of classes. Defined as:

$$C(a) = 2a(1 - a)$$

Both measures help choose splits that lead to more homogeneous (pure) class distributions in child nodes.

- (c) XGBoost improves upon gradient boosting using regularization and second-order optimization. Explain the objective function used in XGBoost. What terms are used for regularization, and what role does the second-order approximation play in training?

Answer: The XGBoost loss function is:

$$\mathcal{L}(f) = \sum_{i=1}^n \ell(y^{(i)}, f(x^{(i)})) + \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2$$

Where:

- ℓ : primary loss (e.g., logistic or squared error),
- J : number of leaves,
- w_j : prediction score at leaf j ,
- γ : penalty for each leaf (controls tree complexity),
- λ : ℓ_2 regularization on leaf weights.

XGBoost uses a second-order Taylor expansion of the loss function (involving both gradient and Hessian) for more accurate and stable optimization during boosting.

- (d) Explain how decision making is performed in stacking. What is the function of the meta-model? Additionally, describe how a gradient boosting model can be incorporated into stacking, and how it learns over time.

Answer: In stacking, multiple base models (level-0 learners) are trained independently on the original training data. Their predictions are then used as features to train a meta-model (level-1 learner), which learns how to combine the base predictions to make final decisions. This allows the ensemble to exploit the strengths of each base model.

Gradient boosting can be used as one of the base learners in stacking. It updates its parameters iteratively by fitting new models to the negative gradient of the loss function (i.e., the residuals). Each new tree improves upon the current ensemble prediction, typically via:

$$f_i(x) = f_{i-1}(x) + \alpha F_i(x)$$

where F_i is the new weak learner and α is the learning rate.

- (e) Explain the bootstrapping process and how it helps create diversity among the models in the ensemble.

Answer: Bootstrapping involves generating new datasets by sampling from the original dataset *with replacement*. Each new dataset (called a bootstrap sample) is used to train one model in the ensemble. Since different subsets are used for each model, they learn different patterns and make different errors. This introduces diversity among models and helps reduce variance when their predictions are aggregated (as in bagging or random forests).

- (f) When generating a bootstrapped sample of size n (equal to the original dataset size), approximately 63.2% of the original datapoints are expected to appear in the sample.

Why does this 63.2% value occur? Derive it mathematically.

How does this sampling method help improve the ensemble's performance, despite each model seeing only a subset of the data?

Answer: Each datapoint has a $1 - \frac{1}{n}$ chance of *not* being selected in a single draw. Over n draws (to create a sample of size n), the probability that a datapoint is never selected is:

$$\left(1 - \frac{1}{n}\right)^n \approx \frac{1}{e} \approx 0.368$$

So the probability that a datapoint is selected at least once is:

$$1 - \frac{1}{e} \approx 0.632$$

This means on average about 63.2% of the original data points appear in a bootstrap sample.

This randomness ensures that each model in the ensemble is trained on a slightly different dataset, which reduces correlation among models and improves generalization when their outputs are combined (e.g., via voting or averaging).